<u>Temática del 2026-2:</u> Fantasías animadas de la historia de Colombia (Abarca eventos locales o universales relevantes para Colombia).

Idea general: Desarrollo de un videojuego sencillo (usando C++ y QT), correspondiente al nivel de conocimientos propio del curso. A pesar de su sencillez, debe estar correctamente desarrollado, ser interesante de jugar (por lo que debe tener un nivel de dificultad adecuado), amigable y agradable a la vista del usuario.
Objetivos: Poner en práctica los conocimientos de: POO, memoria dinámica y GUI.
A pesar de las libertades creativas, delimite apropiadamente su visión del videojuego. En el canal de Youtube del curso puede ver desarrollos previos y tomarlo como guía visual de lo que puede construir.

Momento II
Análisis y diseño de la capa lógica

Fecha de entrega (modalidad presencial): Domingo 25 de Octubre 2026, 11:59 pm.

Fecha de entrega (modalidad virtual): Martes 27 de Octubre 2026, 11:59 pm.

Actividades :

A) **Extensión del contexto**: En un documento de word dentro del directorio del momento I, describa **(en prosa)** la dinámica que plantean para el funcionamiento del juego, inspirada en el guión de la temática elegida.

Plantee **al menos dos niveles**, **cada uno con una dinámica de interacción notablemente diferenciable**. Incluya una sinopsis del contexto correspondiente a cada nivel, y para cada nivel: describa detalladamente las dinámicas, interacciones, vistas, físicas a implementar (tres o más), retos, objetivos y funcionamiento general.

Ejemplo breve:

Idea central : Baseball en la época de las cavernas

- **Nivel** 1: El personaje principal es el bateador y la computadora es el pitcher que le lanza piedras para batear, en una vista cenital. El objetivo es…. los obstáculos o retos son….
- **Nivel 2**: El personaje principal es catcher a quién le van cayendo rocas desde el cielo y debe atraparlas en su guante de caparazón de tortuga, en una vista lateral. En este nivel, ocurren terremotos propio de la época lo cuales afectan la capacidad de moverse del personaje haciéndolo más impreciso (tiembla). El objetivo es…. los obstáculos o retos son….

Nota: La temática de los ejemplos y sprites en este documento puede o no coincidir con la temática requerida específicamente para cada semestre.

Requisitos:

- Las **vistas y dinámicas** de los distintos niveles deben ser significativamente diferentes. Algunas opciones de vista son: plataformero, vista lateral fija, vista con scroll (limitado o ilimitado), cenital (fija o con scroll), entre otros.

- Considere definir tres (o más) **físicas** diferentes (ecuaciones) para implementar el movimiento de los objetos en pantalla (oscilatorios, parabólicos, etc). Hágalo a través de modelos parametrizables desarrollados por ustedes mismos. Distribuya las físicas entre los diferentes niveles y asociales a obstáculos, eventos, características, interacciones o perturbaciones que puedan originar un cambio físico a evidenciar visualmente. Estas físicas deben tener sentido de acuerdo a las reglas del contexto seleccionado. Movimiento rectilíneo no se cuenta como física.

- **Características del personaje**: En el marco de la historia y universo seleccionado por usted, defina al menos 2 características que le den personalidad específica al protagonista y que afecten su comportamiento en algún momento del juego. Por ejemplo: saltarín, rapidín, enojón, payasín, fortachón, agresivo, veloz, pesado, tristón,etc.

Por ejemplo: el personaje principal (capitán cavernícola) al comer una manzana se convierte en fortachón, incrementando su fuerza de bateo por **10 segundos**, sin embargo, esto también lo vuelve más lento al batear en ese periodo.

- Uno de los niveles (o algún evento dentro del nivel) debe estar controlado por tiempo.

- **Componente investigativo**: Incorpore un **agente autónomo SENCILLO** cuyo comportamiento muestre algún nivel de inteligencia dentro del entorno del juego. Este agente puede estar asociado a un personaje o ente de la interfaz y dentro de su diseño debe considerar **necesariamente y por separado** las tareas de:

1) **Percepción**: "atiende, percibe o siente" un estímulo o evento específico
2) **Razonamiento**: interpreta la intención del estímulo y busca la mejor respuesta o acción (toma de decisión).
3) **Acción**: ejecuta una respuesta produciendo una interacción.
4) **Aprendizaje**: almacena y aprende de la información pasada a fin de mejorar sus respuestas luego de cada interacción.

Nota: Este detalle de secciones se debe manifestar explícitamente en el diagrama de clases que se pedirá en (B).

B) Abra un directorio específico para el **Momento II**:

1) Adjunte al menos 4 sprites a utilizar por cada nivel planteado.

2) Diagrama de clases con nivel de detalle medio o superior, especificando relaciones y estructuras de datos a necesitar. Para esta entrega, aborde la capa lógica solamente. Haga uso de herencia (propia) al menos en una ocasión. Modele adecuadamente la arquitectura del agente inteligente.

3) Presente (en diagrama de flujo o en prosa), descripciones en alto nivel para los distintos componentes del agente inteligente a desarrollar.

Ejemplo de sprites:



Ejemplo de Fondos:



Momento I : Contextualización

Fecha de cumplimiento:

a.- Modalidad presencial/flexible: 28 de Septiembre del 2026, 09:00 am.

b.- Modalidad virtual: 29 de Septiembre del 2026, 09:00 am.

Lista de temas por grupo:

https://docs.google.com/document/d/1zdBpzBvEWYqyN3RzLeRPrga7qLholfPG47-r4Hk-luU/edit?usp=sharing

Momento III : Implementación y entrega final

Fecha de entrega: 27 de Noviembre del 2026

Consideraciones para la entrega del proyecto final (entregables), tenerlo muy en cuenta pues la falta de algún requisito se verá reflejado en la evaluación.

- Cumplir el proceso de cada uno de los 3 momentos, y lo indicado en cada uno de ellos.

- **Físicas:** Incluir al menos tres modelos físicos (Movimiento rectilíneo no cuenta).

- POO y memoria dinámica de uso obligatorio.

- Presente el diagrama de clase que incorpore la capa lógica y la capa de GUI (usando QT). Nota: Incluya también una versión resumen del diagrama de clase para su discusión ágil durante la sustentación.Sin DIAGRAMA DE CLASE NO HAY SUSTENTACIÓN.

- Al menos una herencia propia (que no herede de las clases de Qt).

- La noción de dificultad al jugar debe ser parte del desarrollo, debe mostrar evidencia del proceso de abstracción de la dificultad. El videojuego debe ser agradable, usable e interesante para el usuario final. Establezca la opción de configuración de dificultad para uno de los niveles (Esto no puede reducirse a soluciones triviales: recortar el tiempo o subir la cantidad de ítems a coleccionar, etc).

- Puede utilizar contenedores de la STL (o los equivalentes de Qt). Reuso responsable sabiendo justificar las decisiones.

- Gestionar el desarrollo a través de un repositorio usando ramas. Se deben hacer commits de forma regular (al menos uno cada dos días) de tal forma que se evidencie la evolución de la propuesta de solución y su implementación.

- La implementación debe incluir el uso de excepciones en los casos necesarios.

- Se debe implementar usando la interfaz gráfica de QT.

- Generar un trailer (1 min máximo) promocionando el videojuego. El objetivo es que den ganas de jugar luego de verlo. Las tomas deben ser reales, no generadas por IA.

- La documentación incluye un informe siguiendo el modelo de los desafíos previos, incluyendo la versión final de lo entregado en los momentos I y II. Incluya una sección para explicar los cambios que hizo entre lo pautado en los momentos anteriores y la versión final. No se recibirán informes generados por IA.

- Adjuntar un enlace de Youtube a un video que debe incluir lo siguiente:

- Presentación de la solución planteada. Análisis realizado y explicación de la arquitectura del sistema (5 minutos máximo).
- Demostración de funcionamiento del sistema. Explicar cómo funciona: ejemplos demostrativos (5 minutos máximo).
- Explicación del código fuente. Debe justificar cada elección realizada, desde los tipos de variables, las interacciones entre clases, contenedores, etc. Por qué eligió una u otra cosa en cada caso particular y qué ventaja ofrece en comparación con otras que también podrían haberse usado.
- La duración total del video no debe exceder 20 minutos ni ser inferior a 7 minutos.
- Asegúrese que el video tenga buen sonido y que se pueda visualizar con resolución suficiente para apreciar bien los componentes presentados.
- Respetar el tiempo, no exportar acelerado.

- **Generar un archivo ejecutable** del videojuego: ver video en el canal de curso. El código fuente y ejecutable deben estar en el repositorio.

Componentes investigativos:

- Incorporación de al menos 1 agente inteligente (inteligencia simple), según las pautas y estructura previamente indicadas.

- Sonido de fondo (al menos en 1 nivel) + sonido de eventos (incorporar al menos dos). Tener muy presente los derechos de autor para evitar prohibiciones de YouTube.

- Generación del ejecutable (Video disponible en el canal del curso).

Cronograma de entregas:

Entrega final: trailer, video, informe, código y ejecutable. Si no hay: cumplimiento de los momentos I y II, entrega de video, informe y diagrama de clases, no habrá sustentación.



Secciones útiles a considerar para el informe (puede omitir algunas):

- Introducción

  Contexto del proyecto, motivación y público objetivo.

- Planteamiento del problema

  Necesidad que cubre el software y justificación del desarrollo.

- Definición general y objetivos

  Alcance funcional, metas y requisitos principales.

- Especificación de requerimientos

  Requisitos funcionales, requisitos no funcionales, restricciones, dependencias y usuarios destinatarios.

- Metodología y planificación

  Metodología empleada (ágil, cascada, etc.), fases del desarrollo, y cronograma estimado.

- Diseño y arquitectura del sistema

  Explicación jerárquica de los componentes

  Diagrama general de clases, flujo y breves descripciones individuales.

  Dependencias externas y bibliotecas utilizadas

- Desarrollo e implementación

  Breve descripción de la lógica interna, principales algoritmos y decisiones técnicas, sin detallar todo el código fuente.

- Procedimientos de prueba

  Métodos de verificación, casos de prueba, resultados obtenidos y revisión de calidad.

- Guía de instalación y uso

  Instrucciones para instalar y ejecutar el software

  Requisitos técnicos y recomendaciones.

- Resultados y discusión

Funcionamiento logrado, limitaciones actuales, dificultades enfrentadas y mejoras planeadas.

- Conclusiones

  Reflexión sobre el proceso, el aprendizaje obtenido y recomendaciones

- Referencias

  Bibliografía, enlaces y documentación consultada

- Anexos/Apéndices

  Manual de usuario (si aplica), capturas de pantalla, fragmentos de código importantes comentados, diagramas adicionales

FIN DE ESPECIFICACIONES DEL MOMENTO III

.-.-.-. NO PASE: SOLO PARA USUARIOS AUTORIZADOS

.

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]



[redacted]


Idle
Attack Musket
Walk
Attack Bayonet
Run
Jump
Hurt/Hit
Attack Musket
Fall/Die

[redacted]